

(Pages : 4)

N – 8169

Reg. No. :

Name :

Fourth Semester B.Sc./B.C.A. Degree Examination, August 2022

Career Related First Degree Programme under CBCSS

Group 2(b) – Computer Science/Computer Applications

Core Course

CS 1444/CP 1443 : PHP AND MYSQL

(2019 Admission Onwards)

Time : 3 Hours

Max. Marks : 80

SECTION – A

Very Short Answer Type. One word to maximum of one sentence. Answer all questions.

1. _____ or _____ is used to display output in PHP.
2. _____ is the built-in function to find the length of a string in PHP.
3. _____ function is used to set cookie in PHP.
4. _____ function converts a string to all uppercase.
5. What is the output of the following program?

```
<?php
echo "Welcome" . "to" . "PHP";
?>
```
6. _____ function is used to connect with MySQL database.

P.T.O.

7. The information sent from an HTML form using the _____ method is visible to everyone in the browser's address bar.

8. `$salary=array("Sonoo" => "550000", "Vimal" => "250000", "Ratan" => 200000);` is an example for _____ array.

9. _____ function finds the number of items in an indexed array.

10. _____ array represents an array containing one or more arrays in PHP.

(10 × 1 = 10 Marks)

SECTION - B

Short Answer. Not exceeding one paragraph, answer any **eight** questions. Each question carries **2** marks.

11. List the different operators used in PHP.

12. What do you mean by form validation?

13. What is `settype()` in PHP?

14. What do you mean by typecasting?

15. What is the use of `isset()` function?

16. Explain `krsort()` with example program.

17. What is the use of `mysqli_fetch_array()`?

18. What is the difference between open source DBMS and proprietary DBMS? Give example for each.

19. What is `fread()` and `fwrite()` functions in PHP?

20. Mention any four superglobals in PHP.

21. Write a PHP script to delete a table stored in MySQL.

22. Mention the logical operators in MySQL.

23. Differentiate float and double data type in MySQL.
24. Mention any four features of MySQL.
25. Explain the use of LIKE operator in MySQL.
26. What do you mean by INNER JOIN in MySQL? Write the syntax.

SECTION - C

(8 × 2 = 16 Marks)

Short Essay. Not exceeding 120 words, answer any **six** questions. **Each** question carries **4** marks.

27. What are the rules for naming a variable in PHP?
28. Explain break and continue in PHP with examples.
29. Explain with an example how to create functions in PHP.
30. Explain the time() function in PHP.
31. Explain getDate() function in PHP.
32. Discuss various types of cookies.
33. How will you upload a file in PHP?
34. Explain include() function in PHP.
35. Explain \$_FILES in PHP.
36. Explain the purpose of \$_REQUEST.
37. Differentiate asort() and ksort() functions in PHP.
38. Explain how PHP sends mail using the mail() function.

(6 × 4 = 24 Marks)

SECTION – D

Long Essay. Answer any **two** questions. **Each** question carries **15** marks.

39. Explain with examples various data types in PHP.

40. Explain the following with examples.

(a) Cookie management

(b) \$_GET

41. Discuss various loops used in PHP.

42. Write a PHP script to fetch and display the records stored in MySQL.

43. Explain session management in PHP.

44. Explain different types of arrays in PHP.

(2 × 15 = 30 Marks)

No. of Copies	
------------------	--

For Office Use Only

Question Paper Code

N 8169

[Signature]
 Scrutiny 10/6/22

.....
 Remarks on Scrutiny

.....
 No. of pages

Page No.

FOURTH SEMESTER BSc/BCA DEGREE EXAMINATION
CAREER RELATED FIRST DEGREE PROGRAMME UNDER CBCSS
Group2(b)- COMPUTER SCIENCE/COMPUTER APPLICATIONS

Core Course
CS 1444/CP 1443- PHP AND MYSQL
(2018 Admission onwards)

Time: 3 Hours

Answer key

Total Marks: 80

Verified & Approved

[Signature]
 Vellayil N.P.
 Chairman BCA

CS 1444/CP 1443 – PHP and MYSQL

(2018 Admission onwards)

Time: 3 Hours

Maximum Marks: 80

SECTION A [Very Short Answer Type]

[One word to maximum of one sentence. Answer ALL questions]

10 X 1 = 10 marks

1. Echo, print
2. strlen()
3. setcookie()
4. strtoupper()
5. Welcome to PHP
6. mysqli_connect()
7. GET
8. Associative
9. count()
10. multidimensional

SECTION B [Short Answer]

[Not to exceed one paragraph, answer any EIGHT questions. Each question carries TWO marks]

8 X 2 = 16 marks

11. Arithmetic operators, Assignment operators, Comparison operators, Increment/Decrement operators, Logical operators, Bitwise operators, String operators and Array operators.
12. Validation means check the input submitted by the user. There are two types of validation available in PHP. They are as follows. Client-side validation – Validation is performed on the client machine Web browsers. Server-side validation – After the data is submitted, the data has sent to a server and perform validation checks in server machine.
13. The settype() function converts a variable to a specific type. Syntax is settype(variable, type);
14. The meaning of type casting is to use the value of a variable with different data type. Typecasting is a way to utilize one data type variable into the different data type. In PHP variables automatically decide the data type on the basis of the value assignment or context.
15. We can make use of isset() function to check if session variable is already set or not.
16. **ksort()** - Sort associative arrays in descending order, according to the key.
Example Program

```
<?php
$emp=array("name"=>"Smith","dept"=>"Sales","place" =>"Mumbai");
ksort($emp);
echo "Sorted Array in Descending Order of Keys<br>";
print_r($emp)
?>
```
17. The mysqli_fetch_array() function fetches a result row as an associative array, a numeric array, or both.

18. An open source database has code that is open and free for download, modification and re-use. This is the opposite of a proprietary or closed source database in which the code is protected to prevent copying.
19. **fread()** -Read the file's content using fread() function. A new file can be written or text can be appended to an existing file using the PHP **fwrite()** function.
20. \$ GET, \$ _POST, \$ _REQUEST, \$ _SESSION, \$ _COOKIE, \$ _FILES (Any 4)
21.

```
<?php
    $servername = "localhost";
    $username = "username";
    $password = "password";
    $dbname = "myDB";
    // Create connection
    $conn = new mysqli($servername, $username, $password, $dbname);
    // Check connection
    if ($conn->connect_error) {
        die("Connection failed: " . $conn->connect_error);
    } // sql to delete a table
    $sql = "DELETE FROM EMPLOYEE";
    if ($conn->query($sql) === TRUE) {
        echo "Record deleted successfully";
    } else {
        echo "Error deleting record: " . $conn->error;
    } $conn->close();
?>
```
22. AND, OR, and NOT
23. **FLOAT(M,D)** – This is a floating-point number that cannot be unsigned. We can define the display length (M) and the number of decimals (D). This is not required and will default to 10,2, where 2 is the number of decimals and 10 is the total number of digits (including decimals). Decimal precision can go to 24 places for a **FLOAT**. **DOUBLE(M,D)** – A double precision floating-point number that cannot be unsigned. We can define the display length (M) and the number of decimals (D). This is not required and will default to 16,4, where 4 is the number of decimals. Decimal precision can go to 53 places for a **DOUBLE**.
24. Open Source, Scalable, Variety of data types, variety of character sets, secure, support for large databases, client and utility programs.(Any 4)
25. The **LIKE** operator is used in a **WHERE** clause to search for a specified pattern in a column. There are two wildcards often used in conjunction with the **LIKE** operator: The percent sign (%) represents zero, one, or multiple characters. The underscore sign () represents one, single character.
26. The **INNER JOIN** keyword selects records that have matching values in both tables. **INNER JOIN** Syntax - **SELECT** column_name(s) **FROM** table1 **INNER JOIN** table2 **ON** table1.column_name = table2.column_name;

SECTION C [Short Essay]

[Not to exceed 120 words, answer any **SIX** questions. Each question carries **FOUR** marks]

6 X 4 =24 marks

27. A variable starts with the \$ sign, followed by the name of the variable. A variable name must start with a letter or the underscore character. A variable name cannot start with a number. A variable name can only contain alpha-numeric characters and underscores (A-z, 0-9, and _). Variable names are case-sensitive (\$sum and \$SUM are two different variables).
28. The PHP *break* keyword is used to terminate the execution of a loop prematurely. The *break* statement is situated inside the statement block. It gives us full control and whenever we want to exit from the loop we can come out. After coming out of a loop immediate statement to the loop will be executed.

Example Program

```
<?php
$x = 1;
while($x <= 5) {
    echo "The number is: $x <br>";
    $x++;
    if($x==3) break;
}
echo "Executed the break"
?>
```

Output

The number is: 1
The number is: 2
Executed the break

continue Statement

The PHP *continue* keyword is used to halt the current iteration of a loop but it does not terminate the loop. Just like the *break* statement the *continue* statement is situated inside the statement block containing the code that the loop executes, preceded by a conditional test. For the pass encountering *continue* statement, rest of the loop code is skipped and next pass starts.

Example Program

```
<?php
$x = 1;
while($x <= 5) {
    $x++;
    if($x==3) continue;
    echo "The number is: $x <br>";
}
?>
```

29. We can declare and call user-defined functions easily. While creating a function its name should start with keyword function and all the PHP code should be put inside { and } braces. Let's see the syntax to declare user-defined functions.

```
function functionName() {
    code to be executed;
}
```

A function name can start with a letter or underscore but not a number. In PHP, function names are not case-sensitive.

Example Program

```
<html>
  <head>
    <title>Writing PHP Function with Parameters</title>
  </head>
  <body>
    <?php
      function addFunction($a, $b) {
        $sum = $a + $b;
        echo "Sum of the two numbers is: $sum";
      }
      addFunction(100, 200);
    <?>
  </body>
</html>
```

30. PHP's time() function gives you all the information that you need about the current date and time. It requires no arguments but returns an integer. The integer returned by time() represents the number of seconds elapsed since midnight GMT on January 1, 1970. This moment is known as the UNIX epoch, and the number of seconds that have elapsed since then is referred to as a time stamp.
31. The function getdate() optionally accepts a time stamp and returns an associative array containing information about the date. If you omit the time stamp, it works with the current time stamp as returned by time(). Following are the elements contained in the array returned by getdate(). 1 seconds - Seconds past the minutes (0-59). 2 minutes - Minutes past the hour (0 - 59). 3.hours - Hours of the day (0 - 23). 4 mday - Day of the month (1 - 31). 5 wday - Day of the week (0 - 6). 6 - mon - Month of the year (1 - 12). 7 year - Year (4 digits) 8. yday - Day of year (0 - 365). 9 weekday - Day of the week. 10 .month - Month of the year
32. There are two types of cookies, they are: Session Cookie: This type of cookies are temporary and are expire as soon as the session ends or the browser is closed. Persistent Cookie: To make a cookie persistent we must provide it with an expiration time. Then the cookie will only expire after the given expiration time, until then it will be a valid cookie.
33. The user opens the page containing a HTML form featuring a text files, a browse button and a submit button. The user clicks the browse button and selects a file to upload from the local PC. The full path to the selected file appears in the text filed then the user clicks the submit button. The selected file is sent to the temporary directory on the server. The PHP script that was specified as the form handler in the form's action attribute checks that the file has arrived and then copies the file into an intended directory. The PHP script confirms the success to the user.
34. The include() function takes all the text in a specified file and copies it into the file that uses the include function. If there is any problem in loading a file then the include() function generates a warning but the script will continue execution. Including files saves a lot of work. This means that we can create a standard header, footer, or menu file for all our Webpages. Then, when the header needs to be updated, we can only update the header include file.
35. There is one global PHP variable called \$_FILES. This variable is an associate double dimension array and keeps all the information related to uploaded file. So if the value assigned to the input's

- name attribute in uploading form was file, then PHP would create following five variables
- \$_FILES['file']['tmp_name'] - the uploaded file in the temporary directory on the web server.
 - \$_FILES['file']['name'] - the actual name of the uploaded file.
 - \$_FILES['file']['size'] - the size in bytes of the uploaded file.
 - \$_FILES['file']['type'] - the MIME type of the uploaded file.
 - \$_FILES['file']['error'] - the error code associated with this file upload.
36. PHP \$_REQUEST is used to collect data after submitting an HTML form. The PHP \$_REQUEST variable contains the contents of both \$_GET, \$_POST and \$_COOKIE. The PHP \$_REQUEST variable can be used to get the result from form data sent with both the GET and POST methods.
 37. asort() - Sort associative arrays in ascending order, according to the value. ksort() - Sort associative arrays in ascending order, according to the key.
 38. PHP makes use of mail() function to send an email. This function requires three mandatory arguments that specify the recipient's email address, the subject of the message and the actual message additionally there are other two optional parameters. mail(to, subject, message, headers, parameters); to - Required. Specifies the receiver / receivers of the email, subject - Required. Specifies the subject of the email. This parameter cannot contain any newline characters. Message - Required. Defines the message to be sent. Each line should be separated with a LF (\n). Lines should not exceed 70 characters. Headers - Optional. Specifies additional headers, like From, Cc, and Bcc. Parameters - Optional. Specifies an additional parameter to the send mail program. As soon as the mail function is called PHP will attempt to send the email then it will return true if successful or false if it is failed. Multiple recipients can be specified as the first argument to the mail() function in a comma separated list.

SECTION D [Long Essay]

[Answer any TWO questions. Each question carries 15 marks]

2 x 15 = 30 marks

39. PHP supports 8 primitive data types that can be categorized further in 3 types. **Scalar Types, Compound Types and Special Types.** **Scalar Types** - There are 4 scalar data types in PHP. **integer:** They are whole numbers, without a decimal point, like 235. They are the simplest type and they correspond to simple whole numbers, both positive and negative. Integers can be assigned to variables, or they can be used in expressions. Integer can be in decimal (base 10), octal (base 8), and hexadecimal (base 16) format. Decimal format is the default, octal integers are specified with a leading 0, and hexadecimal have a leading 0x. An integer data type is a non-decimal number between -2,147,483,648 and 2,147,483,647. **float:** A float (floating point number) is a number with a decimal point or a number in exponential form. By default, float prints with the minimum number of decimal places needed. **boolean:** They have only two possible values either true or false. PHP provides a couple of constants especially for use as Booleans: TRUE and FALSE. **string:** They are sequences of characters. There are no limits on string length. It depends on the bounds of available memory. We can use both single quoted and double quoted strings. **Compound Types** - There are 2 compound data types in PHP. **array:** An array stores multiple values in one single variable. An array can be declared as follows. \$fruits = array("apple", "orange", "grapes"); **object:** An object is a data type which stores data and information on how to process that data. In PHP, an object must be explicitly declared. First we

must declare a class of object. For this, we use the class keyword. A class is a structure that can contain properties and methods. There are 2 special data types in PHP. **resource:** The special resource type is not an actual data type. It is the storing of a reference to functions and resources external to PHP. A common example of using the resource data type is a database call. **NULL:** Null is a special data type which can have only one value NULL. A variable of data type NULL is a variable that has no value assigned to it. If a variable is created without a value, it is automatically assigned a value of NULL. Variables can also be emptied by setting the value to NULL. The NULL value can be assigned as follows. `$x=null;`

40. a) **Cookie management** - A cookie is a small file with the maximum size of 4KB that the Web server stores on the client computer. Once a cookie has been set, all page requests that follow return the cookie name and value. A cookie can only be read from the domain that it has been issued from. For example, a cookie set using the domain `www.example.com` cannot be read from the domain `career.example.com`. A cookie created by a user can only be visible to them. Other users cannot see its value. Most Web browsers have options for disabling cookies, third party cookies or both. If this is the case then PHP responds by passing the cookie token in the URL. Cookies allow us to track the state of application using small files stored on the user's computer. The path where the cookies are stored depends on the browser. Following is the basic syntax used to create a cookie.

```
<?php
setcookie(cookie_name, cookie_value, [expiry_time], [cookie_path], [domain],
[secure], [httponly]);
?>
```

All parameters except `cookie_name` and `cookie_value` are optional. `setcookie` is the PHP function used to create the cookie. `cookie_name` is the name of the cookie that the server will use when retrieving its value from the `$_COOKIE` array variable. It's mandatory. `cookie_value` is the value of the cookie and its mandatory. `expiry_time` is optional. It can be used to set the expiry time for the cookie such as 1 hour. The time is set using the PHP `time()` functions plus or minus a number of seconds greater than 0 i.e. `time() + 3600` for 1 hour. `cookie_path` is optional. It can be used to set the cookie path on the server. The forward slash "/" means that the cookie will be made available on the entire domain. Sub directories limit the cookie access to the subdomain. `Domain` is optional, it can be used to define the cookie access hierarchy i.e. `www.cookieDomain.com` means entire domain while `www.sub.cookieDomain.com` limits the cookie access to `www.sub.cookieDomain.com` and its sub domains. It's possible to have a subdomain of a subdomain as long as the total characters do not exceed 253 characters. `Secure` is optional, the default is false. It is used to determine whether the cookie is sent via https if it is set to true or http if it is set to false. `Httponly` is optional. If it is set to true, then only client side scripting languages can access them i.e. JavaScript cannot access them.

Example Program

```
<?php
setcookie("user_name","99",time()+ 60,'/'); // expires after 60 seconds
echo 'the cookie has been set for 60 seconds';?>
```

- b) **\$_GET** - The GET method sends the encoded user information appended to the page request. The page and the encoded information are separated by the ? character. The GET method produces a long string that appears in our server logs, in the browser's location. The GET method is restricted to send upto 1024 characters only. Never use GET method if we have password or other sensitive information to be sent to the server. GET can't be used to send binary data, like images or word documents, to the server. The data sent by GET method can be accessed using `QUERY_STRING` environment variable. The PHP provides `$_GET` associative array to access all the sent information using GET method.

Example Program

```
<?php
if( ( $_GET["name"] || $_GET["age"] ) ) {
    echo "Welcome ". $_GET['name']. "<br />";
    echo "You are ". $_GET['age']. " years old.";
    exit();
}
?>
<html>
<body>
<form action = "<?php $_PHP_SELF ?>" method = "GET">
    Name: <input type = "text" name = "name" />
    Age: <input type = "text" name = "age" />
    <input type = "submit" />
</form>
</body>
</html>
```

Output

Name Age

Information sent from a form with the GET method is visible to everyone (all variable names and values are displayed in the URL). GET also has limits on the amount of information to send. GET may be used for sending non-sensitive data. GET should never be used for sending passwords or other sensitive information.

41. PHP supports following four loop types: for loop, while loop, do..while loop and foreach loop

for Loop - The for loop is used when we know how many times we want to execute a statement or a block of statements. The syntax of for loop is as follows.

```
for (initialization; condition; increment){
    code to be executed;
}
```

where the initialization initialize the loop counter value, condition is evaluated for each loop iteration. If it evaluates to TRUE, the loop continues. If it evaluates to FALSE, the loop ends. The increment increases/decreases the loop counter value.

while Loop - The while loop will execute a block of code if and as long as a test expression is true. If the test expression is true then the code block will be executed. After the code has executed the test expression will again be evaluated and the loop will continue until the test expression is found to be false. Following is the syntax of while loop.

```
while (condition) {
    code to be executed;
}
```

do..while Loop - The do..while loop will execute a block of code at least once - it will then repeat the loop as long as a condition is true. The syntax of do..while loop is as follows.

```
do {
    code to be executed;
}
while (condition);
```

foreach Loop - The foreach loop is used to loop through arrays. For each pass, the value of the current array element is assigned to \$value and the array pointer is moved by one and in the next pass, next element will be processed. The syntax of foreach loop is as follows.


```
foreach (array as value) {
    code to be executed;
}
```

```
42. <?php
    $dbhost = 'localhost';
    $dbuser = 'root';
    $dbpass = '';
    $db = 'TESTDB';
    // Create connection
    $conn = mysqli_connect($dbhost, $dbuser, $dbpass, $db);
    // Check connection
    if ($conn->connect_error) {
        die("Connection failed: " . $conn->connect_error);
    }
    echo "Connected successfully";
    $sql = "SELECT emp_id, emp_name, emp_salary FROM employee";
    if (mysqli_query($conn, $sql)) {
        echo "New record created successfully";
    } else {
        echo "Error: " . $sql . "<br>" . mysqli_error($conn);
    }
    $result = mysqli_query($conn, $sql);
    if (mysqli_num_rows($result) > 0) {
        // output data of each row
        while($row = mysqli_fetch_assoc($result)) {
            echo "EMP ID: " . $row["emp_id"] . " - EMP Name: " . $row["emp_name"] . " SALARY: " . $row["emp_salary"] . "<br>";
        }
    } else {
        echo "0 results";
    }
    echo "Fetched data successfully\n";
    mysqli_close($conn);
?>
```

43. A session is a global variable stored on the server. Each session is assigned a unique id which is used to retrieve stored values. Whenever a session is created, a cookie containing the unique session id is stored on the user's computer and returned with every request to the server. If the client browser does not support cookies, the unique php session id is displayed in the URL. Sessions have the capacity to store relatively large data compared to cookies. The session values are automatically deleted when the browser is closed. If we want to store the values permanently, then we should store them in the database. Just like the `$_COOKIE` array variable, session variables are stored in the `$_SESSION` array variable. Just like cookies, the session must be started before any HTML tags. We want to store important information such as the user id more securely on the server where malicious users cannot temper with them. Sometimes we want to pass values from one page to another. There may be situations where we want the alternative to cookies on browsers that do not support cookies. In some case, we want to store global variables in an efficient and more secure

way compared to passing them in the URL. They are useful while developing an application such as a shopping cart that has to temporarily store information with a capacity larger than 4KB. In order to create a session, we must first call the PHP `session_start()` function and then store our values in the `$_SESSION` array variable. Let's suppose we want to know the number of times that a page has been loaded, we can use a session to do that. The code below shows how to create and retrieve values from sessions. We can make use of `isset()` function to check if session variable is already set or not.

Example Program 1

```
<?php
session_start(); //start the PHP_session function
if(isset($_SESSION['page_count']))
{
    $_SESSION['page_count'] += 1;
}
else
{
    $_SESSION['page_count'] = 1;
}
echo 'You are visitor number ' . $_SESSION['page_count'];
?>
```

Output 1

You are visitor number 1

The `session_destroy()` function is used to destroy the whole PHP session variables. If we want to destroy only a session single item, we can use the `unset()` function. The code below illustrates how to use both methods.

Example Program 1

```
<?php
session_destroy(); //destroy entire session
?>
```

Example Program 2

```
<?php
unset($_SESSION['product']); //destroy product session item
?>
```

44. In PHP, there are three types of arrays: **Indexed arrays** - Arrays with a numeric index, **Associative arrays** - Arrays with named keys and **Multidimensional arrays** - Arrays containing one or more arrays. **Associative Arrays** - The associative arrays are very similar to numeric arrays in terms of functionality but they are different in terms of their index. Associative array will have their index as string so that you can establish a strong association between key and values. For example to store the salaries of employees in an array, a numerically indexed array would not be the best choice. Instead, we could use the employee's name as the key in our associative array and the value would be their respective salary. There are two ways to create an associative array. Following program illustrates creating associative arrays.

Example Program

```
<html>
<head>
<title>Associative Arrays</title>
```

```

</head>
<body>
<?php
// Method to create an Associative Array
echo "First Method for Associative Array Creation<br>";
$salary=array("Smith"=>"550000","Johns"=>"250000","Adam"=>"200000");
echo "Smith salary: ".$salary["Smith"]."<br>";
echo "Johns salary: ".$salary["Johns"]."<br>";
echo "Adam salary: ".$salary["Adam"]."<br>";
?>
</body>
</html>

```

Multidimensional Arrays - In a multi-dimensional array, each element in the main array can also be an array. And each element in the sub-array can be an array and so on. Values in the multi-dimensional array are accessed using multiple indices. PHP understands multidimensional arrays that are two, three, four, five, or more levels deep. However, arrays more than three levels deep are hard to manage for most people. PHP multidimensional array is also known as array of arrays. It allows us to store tabular data in an array. PHP multidimensional array can be represented in the form of matrix which is represented by row * column. Following program illustrates how to create a two dimensional array and access its elements.

Example Program

```

<html>
<head>
<title>Multidimensional Arrays</title>
</head>
<body>
<?php
$semp=array ( array(1,"Smith",400000), array(2,"Johns",500000), array(3,"Adam",300000));
echo $semp[0][0].".Name: ".$semp[0][1].", Salary: ".$semp[0][2]."<br>";
echo $semp[1][0].".Name: ".$semp[1][1].", Salary: ".$semp[1][2]."<br>";
echo $semp[2][0].".Name: ".$semp[2][1].", Salary: ".$semp[2][2]."<br>";
?>
</body>
</html>

```